Lecture Slide – 16
Runge-Kutta Method

Theory & Derivation

RK2 & RK4 Methods

Worked Examples

Euler's Method — The Foundation

Euler's Formula: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$ where h = step size

Derivation (Taylor Series)

From Taylor expansion:

$y(x+h) = y(x) + h \cdot y'(x) + (h^2/2) \cdot y''(x) + \ldots$

Keeping only the first-order term:

$y_{n+1} \approx y_n + h \cdot f(x_n, y_n)$

Truncation Error = $O(h^2)$ per step → $O(h)$ global

Limitations of Euler's Method

- ✗ Only 1st-order accurate (O(h) globally)
- ✗ Large step sizes lead to significant error
- ✗ Error accumulates rapidly over many steps
- ✗ Unstable for stiff equations

Solution: Use multiple slope evaluations per step to achieve higher-order accuracy — this is the Runge-Kutta idea.

The Runge-Kutta Idea — Weighted Average of Slopes

General RK Framework: $y_{n+1} = y_n + h \cdot$ (weighted average of slopes $k_1, k_2, \ldots, k_s$)

k_1	k_2	k_3	k_4
Slope at start of step	Slope at midpoint (using k_1)	Slope at midpoint (using k_2)	Slope at end of step

Weighted Combination

$y_{n+1} = y_n + h \cdot (w_1 k_1 + w_2 k_2 + w_3 k_3 + w_4 k_4)$ where $w_1 + w_2 + w_3 + w_4 = 1$

The weights (w_i) and evaluation points are chosen to match as many terms of the Taylor series as possible → higher order accuracy.

Order of accuracy = number of Taylor series terms matched → More slopes = Higher order = More accurate

Second-Order Runge-Kutta (RK2) — Derivation

Theoretical Derivation

Start with general 2-stage form:

$y_{n+1} = y_n + h(w_1k_1 + w_2k_2)$

where:

$k_1 = f(x_n, y_n)$

$k_2 = f(x_n + \alpha h, y_n + \beta h k_1)$

Expand k_2 via Taylor series in two variables:

$k_2 \approx f + \alpha h f_x + \beta h k_1 f_y + O(h^2)$

Matching with 2nd-order Taylor expansion gives:

$w_1 + w_2 = 1$

$w_2\alpha = 1/2$

$w_2\beta = 1/2$

One solution (Heun's Method): $w_1=w_2=1/2$, $\alpha=\beta=1$

RK2 (Heun's Method) — Final Form

Stage 1

$$k_1 = h \cdot f(x_n, y_n)$$

Stage 2

$$k_2 = h \cdot f(x_n + h,\ y_n + k_1)$$

Update

$$y_{n+1} = y_n + (k_1 + k_2) / 2$$

Global Error: $O(h^2)$ | 2 function evaluations per step

Fourth-Order Runge-Kutta (RK4) — The Gold Standard

RK4 Principle: Evaluate the slope at 4 strategic points and take a weighted average so that the update matches the Taylor series through the $O(h^5)$ term, giving $O(h^4)$ global accuracy.

k_1 — Slope at start of interval

$$k_1 = h \cdot f(x_n, y_n)$$

(at (x_n, y_n))

k_2 — Slope at midpoint using k_1

$$k_2 = h \cdot f(x_n + h/2,\ y_n + k_1/2)$$

(midpoint estimate 1)

k_3 — Slope at midpoint using k_2

$$k_3 = h \cdot f(x_n + h/2,\ y_n + k_2/2)$$

(midpoint estimate 2)

k_4 — Slope at end using k_3

$$k_4 = h \cdot f(x_n + h,\ y_n + k_3)$$

(at $(x_{n+1}, \sim y_{n+1})$)

Final Update: $y\_\{n+1\} = y_n + (k_1 + 2k_2 + 2k_3 + k_4) / 6$ Global Error: $O(h^4)$ | 4 evaluations/step

Why the Weights 1, 2, 2, 1 in RK4?

Simpson's Rule Analogy: The weights (1, 2, 2, 1)/6 **mirror Simpson's 1/3 rule for numerical integration**. The midpoint slopes (k_2, k_3) are given double weight because they capture the curvature in the interior of the interval — they carry more information than the endpoints.

k_1

weight = 1/6

Start-point slope. Used once as initial direction.

k_2

weight = 2/6

Mid-point slope (improved). Double weight — captures early curvature.

k_3

weight = 2/6

Mid-point slope (refined). Double weight — more accurate interior estimate.

k_4

weight = 1/6

End-point slope. Used once as final direction check.

In Simpson's Rule

$$\text{Average} \approx \frac{\text{Start} + 4(\text{Middle}) + \text{End}}{6}$$

The Translation to RK4:

- The single "Middle" in Simpson's Rule is split into **two** slopes (k_2 and k_3) because the slope depends on both time AND the changing solution.
- The weight of 4 is split equally: **2 for k_2 and 2 for k_3.**

Accuracy Comparison

Accuracy Comparison

Method	Stages	Local Error	Global Error
Euler	1	$O(h^2)$	$O(h)$
RK2 (Heun)	2	$O(h^3)$	$O(h^2)$
RK4 (Classic)	4	$O(h^5)$	$O(h^4)$

EXAMPLE 1 — Applying RK4 to a Simple ODE

Problem: Solve $dy/dx = x + y$, $y(0) = 1$ using RK4 with step size $h = 0.2$. Compute $y(0.2)$.

Exact solution: $y = 2e^x - x - 1$

k_1: $k_1 = h \cdot f(x_0, y_0) = 0.2 \cdot f(0, 1)$

$= 0.2 \cdot (0 + 1) = 0.2000$

k_2: $k_2 = h \cdot f(0 + h/2, 1 + k_1/2) = 0.2 \cdot f(0.1, 1.1)$

$= 0.2 \cdot (0.1 + 1.1) = 0.2400$

k_3: $k_3 = h \cdot f(0 + h/2, 1 + k_2/2) = 0.2 \cdot f(0.1, 1.12)$

$= 0.2 \cdot (0.1 + 1.12) = 0.2440$

k_4: $k_4 = h \cdot f(0 + h, 1 + k_3) = 0.2 \cdot f(0.2, 1.244)$

$= 0.2 \cdot (0.2 + 1.244) = 0.2888$

Final Calculation

$y(0.2) = 1 + (k_1 + 2k_2 + 2k_3 + k_4)/6 = 1 + (0.2 + 0.48 + 0.488 + 0.2888)/6$

$y(0.2) \approx 1.2428$ | **Exact: $y(0.2) = 2e^{0.2} - 0.2 - 1 \approx 1.2428$** | **Error ≈ 0.000015**

EXAMPLE 2 — Newton's Law of Cooling via RK4

Problem: A cup of coffee at 90°C cools in a room at 20°C. Cooling rate k = 0.1 min^{-1}.
ODE: $dT/dt = -k(T - T_a) = -0.1(T - 20)$, $T(0) = 90$. Use RK4 with h = 1 min to find T(1).
Exact: T(t) = 20 + 70·e^(−0.1t)

k_1: $k_1 = h \cdot f(0, 90) = 1 \cdot (-0.1)(90 - 20)$

$= -7.0000$

k_2: $k_2 = h \cdot f(0.5, 90 + k_1/2) = 1 \cdot (-0.1)(83 - 20)$

$= -6.3000$

k_3: $k_3 = h \cdot f(0.5, 90 + k_2/2) = 1 \cdot (-0.1)(86.85 - 20)$

$= -6.6850$

k_4: $k_4 = h \cdot f(1, 90 + k_3) = 1 \cdot (-0.1)(83.315 - 20)$

$= -6.3315$

Final Calculation

$T(1) = 90 + (k_1 + 2k_2 + 2k_3 + k_4) / 6$
$= 90 + (-7.0 + 2(-6.3) + 2(-6.685) + (-6.3315)) / 6 = 90 + (-39.3015)/6$
T(1) ≈ 83.45°C | Exact: T(1) = 20 + 70·e^(−0.1) ≈ 83.33°C | Error ≈ 0.12°C ✓

EXAMPLE 3 — Solving a 2nd-Order ODE with RK4

Problem Statement

$y'' + 0.6\,y' + 8y = 0$ with $y(0) = 4,\ y'(0) = 0$

Solve from x = 0 to x = 5 using RK4 with step size h = 0.5

Key Insight: RK4 works on FIRST-order ODEs only. We must convert this 2nd-order ODE into a system of two coupled 1st-order ODEs by introducing a new variable.

Step 1: Introduce New Variables

Let: $y_1 = y$ and $y_2 = y'$

Then by definition:

$y_1' = y_2$ … (i)

From the original ODE:

$y'' = -0.6y' - 8y$

$y_2' = -0.6y_2 - 8y_1$ … (ii)

Step 2: Rewrite as a System

System of 1st-order ODEs:

$f_1(x, y_1, y_2) = y_2$

$f_2(x, y_1, y_2) = -0.6y_2 - 8y_1$

Initial conditions:

$y_1(0) = 4,\ y_2(0) = 0$

This technique generalises: any n-th order ODE can be converted to n coupled 1st-order ODEs and solved with RK4.

EXAMPLE 3 — RK4 System Formulas & First Step (x = 0 → 0.5)

RK4 Applied to a 2-Variable System (y_1, y_2 updated simultaneously each step)

$k_{i,}y_1 = h \cdot f_1(x, y_1, y_2)$ $k_{i,}y_2 = h \cdot f_2(x, y_1, y_2)$ → $y_{1n+1} = y_{1n} + (k_1y_1 + 2k_2y_1 + 2k_3y_1 + k_4y_1)/6$ $y_{2n+1} = y_{2n} + (k_1y_2 + 2k_2y_2 + 2k_3y_2 + k_4y_2)/6$

Stage k_1 *(Slopes at ($x=0$, $y_1=4$, $y_2=0$))*

$k_1y_1 = 0.5 \times f_1(0, 4, 0) = 0.5 \times 0 = 0.0000$

$k_1y_2 = 0.5 \times f_2(0, 4, 0) = 0.5 \times (-0.6 \times 0 - 8 \times 4) = -16.0000$

Stage k_2 *(Midpoint using k_1)*

$k_2y_1 = 0.5 \times f_1(0.25, 4+0/2, 0+(-16)/2) = 0.5 \times (-8) = -4.0000$

$k_2y_2 = 0.5 \times f_2(0.25, 4, -8) = 0.5 \times (-0.6 \times (-8) - 8 \times 4) = -13.6000$

Stage k_3 *(Midpoint using k_2)*

$k_3y_1 = 0.5 \times f_1(0.25, 4+(-4)/2, 0+(-13.6)/2) = 0.5 \times (-6.8) = -3.4000$

$k_3y_2 = 0.5 \times f_2(0.25, 2, -6.8) = 0.5 \times (-0.6 \times (-6.8) - 8 \times 2) = -5.9600$

Stage k_4 *(End-point using k_3)*

$k_4y_1 = 0.5 \times f_1(0.5, 4+(-3.4), 0+(-5.96)) = 0.5 \times (-5.96) = -2.9800$

$k_4y_2 = 0.5 \times f_2(0.5, 0.6, -5.96) = 0.5 \times (-0.6 \times (-5.96) - 8 \times 0.6) = -0.6120$

$y_1(0.5) = 4 + (0 + 2(-4) + 2(-3.4) + (-2.98))/6$

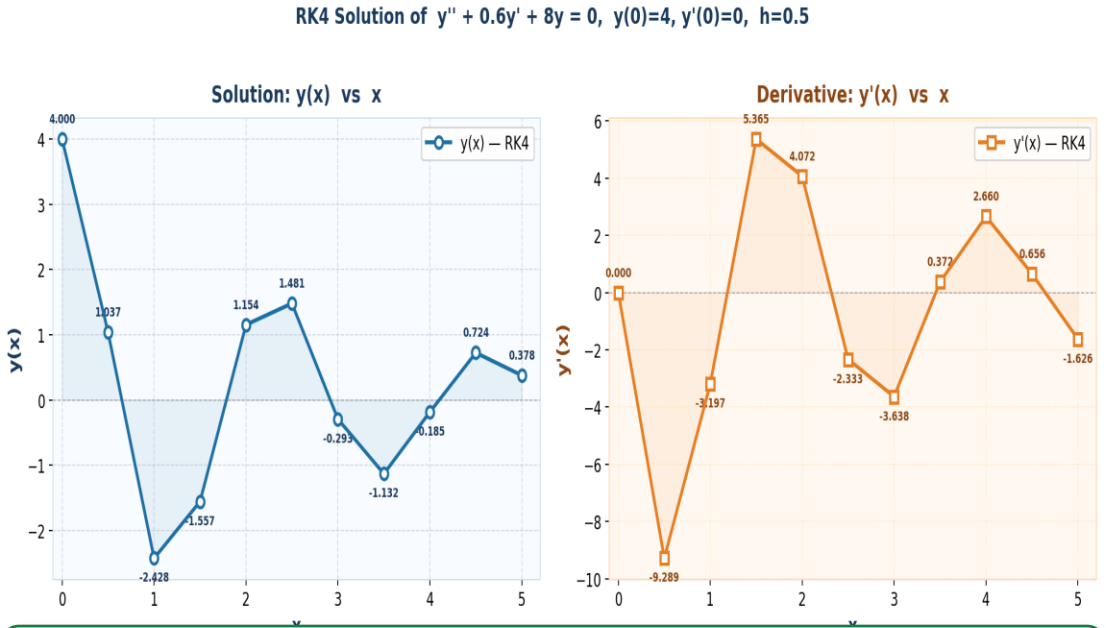
$= 4 + (-18.78)/6 = 4 - 3.130 = 1.0367$ ✓ | $y_2(0.5) = 0 + (-16 + 2(-13.6) + 2(-5.96) + (-0.612))/6 = -9.2887$

EXAMPLE 3 — Complete Results Table & Solution Plot

RK4 Results (h = 0.5)

x	y(x)	y'(x)
0.0	4.0000	0.0000
0.5	1.0367	−9.2887
1.0	−2.4276	−3.1969
1.5	−1.5571	5.3654
2.0	1.1539	4.0719
2.5	1.4810	−2.3334
3.0	−0.2935	−3.6375
3.5	−1.1319	0.3723
4.0	−0.1853	2.6602
4.5	0.7241	0.6564
5.0	0.3782	−1.6258

Oscillation: Natural freq ω = √8 ≈ 2.83 rad



Physical Interpretation

The equation models a damped oscillator (spring-mass system). The damping coefficient (0.6) causes the oscillations to gradually decrease in amplitude — confirmed by the plot showing decaying oscillations approaching zero as x → 5.

Error Analysis & Step Size Selection

Types of Error

1. Local Truncation Error (LTE)
Error introduced in a single step. For RK4: $O(h^5)$.

2. Global Error
Accumulated error over all steps. For RK4: $O(h^4)$.

3. Round-off Error
Due to finite machine precision. Grows as $h \to 0$.

Choosing Step Size h

- Smaller h → smaller truncation error
- Smaller h → more steps → more computation
- Smaller h → larger accumulated round-off
- Optimal h balances truncation & round-off
- Adaptive RK: automatically adjusts h based on estimated local error (e.g. RK45 / Dormand-Prince)

Runge-Kutta Family — Overview

Method	Stages	Order	Local Error	Best Use
RK1 = Euler	1	1st	$O(h^2)$	Teaching / very rough
RK2 (Heun/Midpoint)	2	2nd	$O(h^3)$	Moderate accuracy
RK3	3	3rd	$O(h^4)$	Rarely used in practice
RK4 (Classic)	4	4th	$O(h^5)$	General engineering/science
RK45 (Dormand-Prince)	6	4th/5th	$O(h^5)$	Adaptive, most popular

Summary — Runge-Kutta Method

1 Core Principle

RK methods evaluate the derivative f(x, y) at multiple points within one step and combine them as a weighted average to achieve higher-order Taylor series accuracy.

2 RK4 Formula

$y_{n+1} = y_n + (k_1 + 2k_2 + 2k_3 + k_4)/6$ with 4 function evaluations per step. Global error $O(h^4)$. The double weight on k_2, k_3 reflects their interior (midpoint) importance.

3 Accuracy vs. Cost

RK4 is 4× more expensive than Euler per step but achieves massively better accuracy. For the same error tolerance, RK4 needs far fewer steps overall.

4 Practical Use

RK4 is the standard workhorse for non-stiff ODEs. For adaptive error control, Dormand-Prince RK45 is widely used in MATLAB (ode45), SciPy (RK45), and engineering software.